CHARUSAT®
अमृतं तु विद्या
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical-10.1

Github Link: https://github.com/rudraaaajoshi/FDSA_BATCH_A/tree/main/Practical-10

Difficulty:

Understanding collision handling and moving to the next slot is slightly challenging.

Error:

If the assigned slot is already occupied, directly inserting the vehicle would overwrite the existing vehicle. When the lot is full, the search could keep looping forever.

Solution:

Use linear probing. If the calculated slot is occupied, move to the next slot (index + 1) % 10 until an empty slot is found. Check whether all 10 slots have been searched to avoid an infinite loop.

Outcome:

All vehicles are placed correctly without overwriting existing vehicles. If the parking lot is completely full, the program safely reports that no slot is available.

Code(Github) Link:

https://github.com/rudraaaajoshi/FDSA_BATCH_A/blob/main/Practical-10/pr-10.1.cpp

Output:

```
Enter number of vehicles: 5
Enter registration number: 123
Enter registration number: 456
Enter registration number: 789
Enter registration number: 147
Enter registration number: 369

Final Parking Slots:
Slot 0 : 369
Slot 1 : Empty
Slot 2 : Empty
Slot 3 : 123
Slot 4 : Empty
Slot 5 : Empty
Slot 6 : 456
Slot 7 : 147
Slot 8 : Empty
Slot 9 : 789
```

Key Question:

Problem-1 If five consecutive vehicle registrations all end in the same digit, trace where each one gets placed in the parking lot. Now explain why this clustering effect is a structural weakness of this collision strategy — not just for this input, but in general.

Answer:

This is called **clustering**. Linear probing always checks the next slot, so vehicles with the same hash value form a continuous group. This is a structural weakness because collisions repeatedly create long blocks, increasing search and insertion time.

Practical-10.2

Difficulty:

Understanding how multiple books are stored on the same shelf is slightly challenging.

Error:

If multiple books get the same shelf number, storing only one book would lose the previous book.

Solution:

Use chaining. Each shelf has a linked list, so books with the same code % 10 are added to the same shelf.

Outcome:

All books are stored without overwriting. As more books are added to the same shelf, the search time increases because more books need to be checked.

Code(Github) Link:https://github.com/rudraaaajoshi/FDSA_BATCH_A/blob/main/Practical-10/pr-10.2.cpp

Output:

```
Enter number of books: 3
Enter book code: 101
Enter book code: 300
Enter book code: 102

Final Shelf Contents:
Shelf 0: 300
Shelf 1: 101
Shelf 2: 102
Shelf 3: Empty
Shelf 4: Empty
Shelf 5: Empty
Shelf 6: Empty
Shelf 7: Empty
Shelf 8: Empty
Shelf 9: Empty
```

Key Question:

Problem-2

Suppose all book codes in Problem 2 are multiples of 10. Without tracing through the code, predict what the final shelf layout would look like and explain why. What does this tell you about the relationship between the hash function and the input data?

Answer:

This shows that the performance of a hash table depends on the **hash function and the input data**. A poor match between them can cause many collisions and make searching slower.

Practical-10.3

Difficulty:

Understanding the two hash functions and the jump size during collisions is slightly challenging.

Error:

If the jump size is a factor of the table size, the program may visit only some slots and fail to find an available slot, even when other slots are empty.

Solution:

Use double hashing. The first hash gives the starting slot, and the second hash calculates the jump size whenever a collision occurs.

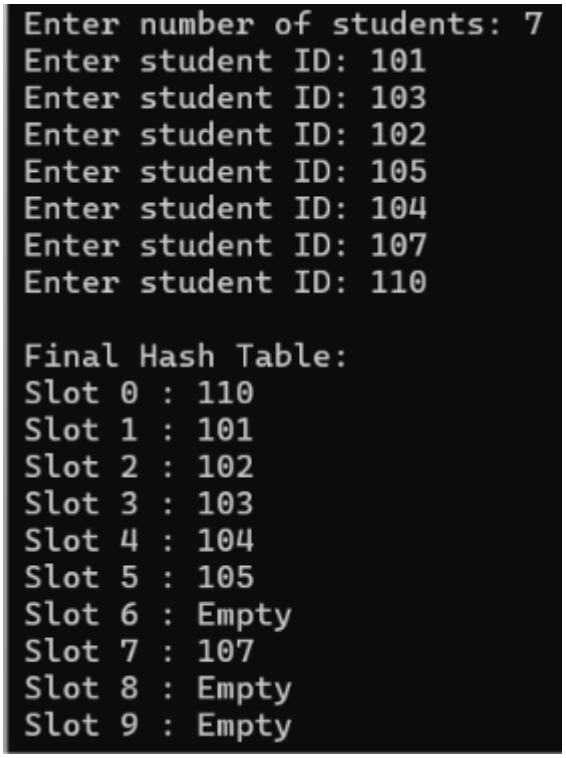
Outcome:

Collisions are spread more evenly because different IDs can have different jump sizes. Unlike linear probing, it does not always move to the next slot, reducing clustering.

Code(Github) Link:

https://github.com/rudraaaajoshi/FDSA_BATCH_A/blob/main/Practical-10/pr-10.3.cpp

Output:



Key Question:

Problem-3 what would happen if the second rule always produced a jump size of zero? Trace through the consequence and explain how this failure is related to the choice of the second hash function rather than the data itself.

Answer:

The problem is with the **second hash function**, not the student IDs. A valid second hash function must produce a non-zero jump so that the program can move to other slots and find an empty position.